В первом блоке мы создали приложение на flask и упаковали его в докер, а именно: в файле app.py написали код приложения на Python. Это приложение принимает запросы от пользователей и возвращает ответы. В файле requirements.txt указали библиотеку Flask, чтобы она автоматически устанавливалась. В файле Dockerfile написан код, который копирует эти файлы в контейнер. Но этот файл он плохой, поэтому его готовый размер будет большим. Благодаря докеру программа упаковалась вместе со всеми библиотеками, что удонбо, потому что можно сделать один такой докер и удобно использовать на каждом компьютере. Это сокращает время.

Контрольный вопрос 1: Образ большой, потому что мы упаковали в него python:3.12, в котором есть ненужные для созданного приложения файлы, программы, инструменты.

Во второй части мы переделали предыдущий образ, чтобы он был правильнее и меньше весил. В файл .dockerignore добавили то, что не надо упаковывать. И при запуске выделили контейнеру определенное количество памяти и процессора.

Команда docker images myapp показала разницу между плохим и хорошим образом: myapp:bad занимает 1.62GB на диске, а myapp:good — всего 81.9MB.

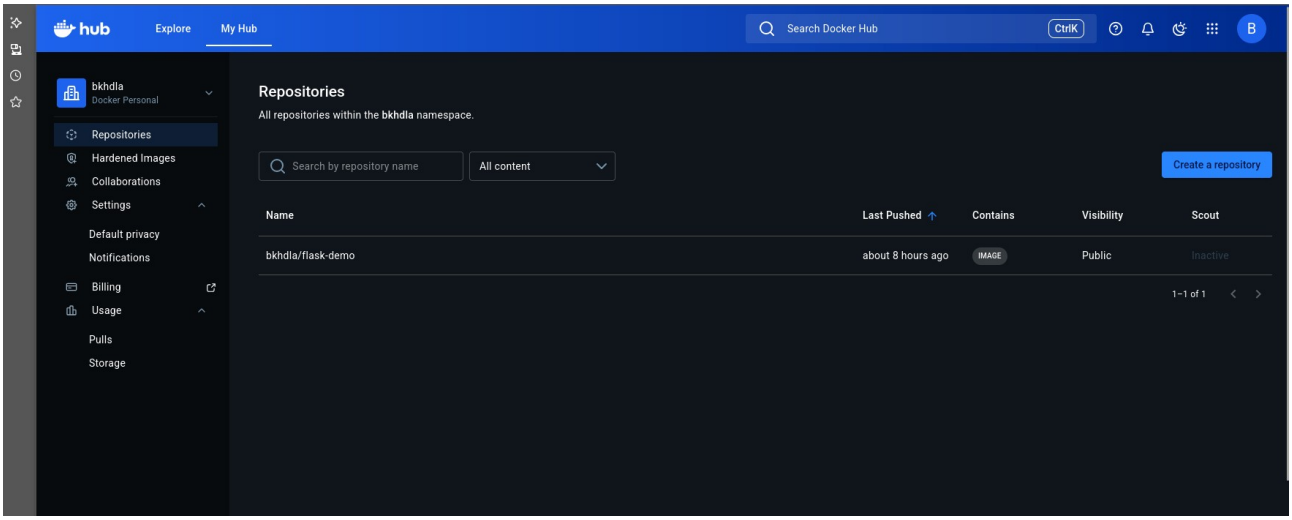
```
daniela@fedora:~/docker-lab$ sudo docker images myapp
IMAGE        ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:bad    fdf34270071d       1.62GB          416MB
myapp:good   de28cc0184df       81.9MB         19.6MB
daniela@fedora:~/docker-lab$
```

Docker состоит из слоев. Они нужны когда нпример требуется поменять одну команду и вместо того, чтобы собирать докер заново, можно просто пересобрать слои после измененной команды. Для анализа слоев мы использовали команды docker history myapp:good и docker history myapp:ba. Они показали колько команд выполнялось, когда и какой размер занимает каждый слой. На скриншоте видно, что у хорошего докера слои, которые занимают больше вмего места - это установка пакетов через apk add.

```
^Cdaniela@fedora:~/docker-lab$ sudo docker history myapp:good
IMAGE          CREATED        CREATED BY                                      SIZE      COMM
272b8adbd6f6   2 minutes ago  CMD ["python" "app.py"]                         0B        buil
ockerfile.v0
<missing>      2 minutes ago  ENV APP_VERSION=2.0                             0B        buil
ockerfile.v0
<missing>      2 minutes ago  COPY app.py . # buildkit                        4.1kB     buil
ockerfile.v0
<missing>      2 minutes ago  COPY /usr/local/lib/python3.12/site-packages…   16.7MB    buil
ockerfile.v0
<missing>      7 minutes ago  WORKDIR /app                                    0B        buil
ockerfile.v0
<missing>      3 weeks ago    CMD ["python3"]                                 0B        buil
ockerfile.v0
<missing>      3 weeks ago    RUN /bin/sh -c set -eux;  for src in idle3 p…   20.5kB    buil
ockerfile.v0
<missing>      3 weeks ago    RUN /bin/sh -c set -eux;   apk add --no-cach…   43.2MB    buil
ockerfile.v0
<missing>      3 weeks ago    ENV PYTHON_SHA256=c08bc65a81971c1dd578318282…   0B        buil
ockerfile.v0
<missing>      3 weeks ago    ENV PYTHON_VERSION=3.12.13                      0B        buil
ockerfile.v0
<missing>      3 weeks ago    ENV GPG_KEY=7169605F62C751356D054A26A821E680…   0B        buil
ockerfile.v0
<missing>      3 weeks ago    RUN /bin/sh -c set -eux;  apk add --no-cache…   3.62MB    buil
ockerfile.v0
<missing>      3 weeks ago    ENV LANG=C.UTF-8                                0B        buil
ockerfile.v0
<missing>      3 weeks ago    ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buil
ockerfile.v0
<missing>      8 weeks ago    CMD ["/bin/sh"]                                 0B        buil
ockerfile.v0
<missing>      8 weeks ago    ADD alpine-minirootfs-3.23.3-x86_64.tar.gz /…   10.1MB    buil
ockerfile.v0
daniela@fedora:~/docker-lab$
```

Утилита dive красиво отображает все слои докера и показывает какие из них занимают много места. Также мы посмотрели какие файлы вообще лежат в образе с помощью команды tar, которая вывела на экран все файлы, вытащенные из временного контейнера.

Команда docker stats показывает, сколько ресурсов компьютера использует активный докер.

```
daniela@fedora:~ — sudo nano /etc/docker/daemon.json    daniela@fedora:~/docker-lab — sudo docker stats app-
CONTAINER ID   NAME       CPU %     MEM USAGE / LIMIT   MEM %     NET I/O         BLOCK I/O
S
662d67ace664   app-good   0.02%     22.62MiB / 128MiB   17.68%    3.46kB / 776B   0B / 3.92MB
```

В последнем пункте мы опубликовали докер, чтобы его можно было использовать на любом компьютере. Сначала через команду docker login мы вошли в свой аккаунт на Docker Hub. Потом переименовали myapp:good на название с именем пользователя и версией. Команда docker push загрузила образ на облако. Для проверка мы удалили докер на хосте и скачали его заново с docker hub.